



UNIDAD 1

Algoritmos Bioinspirados.

TEMA 2:

Ejemplos Clásicos y Aplicaciones.

Descripción:

Esta unidad presenta los fundamentos de los **algoritmos Bioinspirados**, clasificándolos según sus características y origen natural.

Abordan modelos inspirados en procesos biológicos, físicos y ecológicos, junto a sus motivaciones científicas.

Se analizarán ejemplos representativos como los **Algoritmos Genéticos** (GA), la **Optimización por Enjambre de Partículas** (PSO) y la **Optimización de Colonias de Hormigas** (ACO).

Para cada uno, se evalúan sus aplicaciones comunes y se compara su desempeño con otras **metaheurísticas**. Finalmente, se busca que el análisis de estos enfoques proporcione una visión crítica que permita comprender su utilidad en la resolución de problemas complejos y en la investigación de nuevas soluciones.

Metaheurísticas

PERIODO ACADÉMICO 2026-1S

TABLA DE CONTENIDO

Tema 2. Ejemplos Clásicos y Aplicaciones.	2
2.1. Algoritmos Bioinspirados PSO (Particle Swarm Optimization).....	2
2.2. Algoritmos Bioinspirados GA (Genetic Algorithms) y ACO (Ant Colony Optimization).....	6
2.3. Aplicaciones Comunes de los Algoritmos Bioinspirados.....	17
2.4. Comparación preliminar de desempeño.....	19
Bibliografía	21

Tema 2. Ejemplos Clásicos y Aplicaciones.

A continuación, se describen tres de los ejemplos más representativos.

2.1. Algoritmos Bioinspirados PSO (Particle Swarm Optimization).

La Optimización por Enjambre de Partículas (PSO) está inspirada en el comportamiento colectivo observado en la naturaleza:

- **Aves en bandada:** buscan alimento moviéndose como un grupo coordinado.
- **Peces en cardumen:** se mueven en patrones organizados para evitar depredadores o encontrar rutas más cortas hacia los recursos.

El principio fundamental es que un grupo de agentes simples (partículas) puede alcanzar soluciones complejas mediante interacción social y adaptación individual.

Estructura Básica del Algoritmo PSO

Inicialización

Se genera una población de partículas, cada una con una posición (solución candidata) y una velocidad.

Evaluación

Cada partícula se evalúa mediante una función objetivo que mide la calidad de la solución.

Actualización de memoria individual y colectiva

- Cada partícula guarda su mejor posición personal (pBest).
- Se identifica la mejor posición global (gBest) alcanzada por todo el grupo.

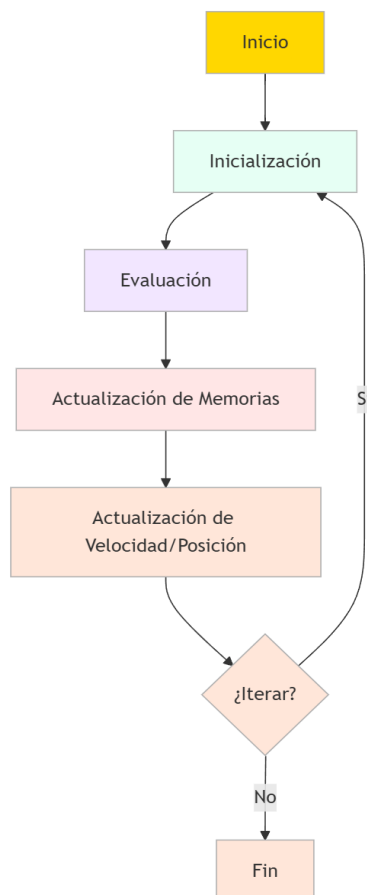
Actualización de velocidad y posición

La velocidad se ajusta considerando tres factores:

- **Inercia (w):** mantiene la dirección actual.
- **Componente cognitiva ($c1$):** atrae hacia la mejor experiencia individual.
- **Componente social ($c2$):** atrae hacia la mejor experiencia del grupo.

Figura 1

Diagrama de flujo del algoritmo de Optimización por Enjambre de Partículas (PSO)



Fuente: Elaboración Propia

Fórmulas generales en PSO

Velocidad

Es el vector que indica cómo cambiará la posición en la siguiente iteración este controla dirección y magnitud del movimiento, si la posición cambia en cada paso, es porque la velocidad "arrastra" a la partícula hacia nuevas regiones.

$$v_i = (v_{i1}, v_{i2})$$

$$v_i(t+1) = w \cdot v_i(t) + c_1 \cdot r_1 \cdot (pBest_i - x_i(t)) + c_2 \cdot r_2 \cdot (gBest - x_i(t))$$

Donde

- **Inercia**

La inercia permite que la partícula "recuerde" y mantenga temporalmente la dirección que le ha dado buenos resultados, balanceando la exploración del espacio de búsqueda.

$$w \cdot v_i(t)$$

- **Cognitivo**

Este componente permite que cada partícula aprenda de su trayectoria personal, guiándose por sus propias experiencias positivas en la búsqueda de soluciones.

$$c_1 \cdot r_1 \cdot (pBest_i - x_i)$$

- **Social**

Este componente permite que el enjambre comparta información y converja eficientemente hacia las regiones más promisorias del espacio de búsqueda, aprovechando el conocimiento grupal.

$$c_2 \cdot r_2 \cdot (gBest - x_i)$$

Posición

Es el punto actual de la partícula en el espacio de búsqueda este representa una solución candidata al problema, si estamos optimizando una función de 2 variables $f(x,y)$, la posición de la partícula podría ser:

$$x_i = (x_{i1}, x_{i2})$$

Relación entre la posición actual y la velocidad para calcular la actualización de posición.

$$x_i(t + 1) = x_i(t) + v_i(t + 1)$$

Ejemplo de Implementación Ejemplo en Python

```
1 import numpy as np
2
3 # Parámetros
4 n_particles = 30
5 n_iterations = 100
6 w, c1, c2 = 0.7, 1.5, 1.5
7
8 # Inicialización
9 dim = 2
10 X = np.random.uniform(-10, 10, (n_particles, dim))
11 V = np.random.uniform(-1, 1, (n_particles, dim))
12
13 # Mejor posición individual y global
14 pbest = X.copy()
15 pbest_val = np.array([np.sum(x**2) for x in X])
16 gbest = X[np.argmin(pbest_val)]
17 gbest_val = np.min(pbest_val)
18
19 # Iteraciones
20 for t in range(n_iterations):
21     for i in range(n_particles):
22         fitness = np.sum(X[i]**2)
23         if fitness < pbest_val[i]:
24             pbest[i] = X[i]
25             pbest_val[i] = fitness
26         if fitness < gbest_val:
27             gbest = X[i]
28             gbest_val = fitness
29
30     # Actualización de velocidad y posición
31     r1, r2 = np.random.rand(n_particles, dim), np.random.rand(n_particles, dim)
32     V = w*V + c1*r1*(pbest - X) + c2*r2*(gbest - X)
33     X = X + V
34
35 print("Mejor solución encontrada:", gbest)
36 print("Valor de la función:", gbest_val)
```

Fuente: Elaboración Propia

Ejemplo

Implementación en R

```
1
2  install.packages("pso")
3
4  library(pso)
5
6  # Definición de la función
7  sphere <- function(x) {
8    sum(x^2)
9  }
10
11 # Ejecutar PSO
12 set.seed(123)
13 result <- psoptim(par = c(5,5), fn = sphere, lower = c(-10, -10), upper = c(10, 10),
14                  control = list(maxit = 100, s = 30))
15
16 print(result$par) # Mejor posición encontrada
17 print(result$value) # Valor de la función
18
```

Fuente: Elaboración Propia

2.2. Algoritmos Bioinspirados GA (Genetic Algorithms) y ACO (Ant Colony Optimization).

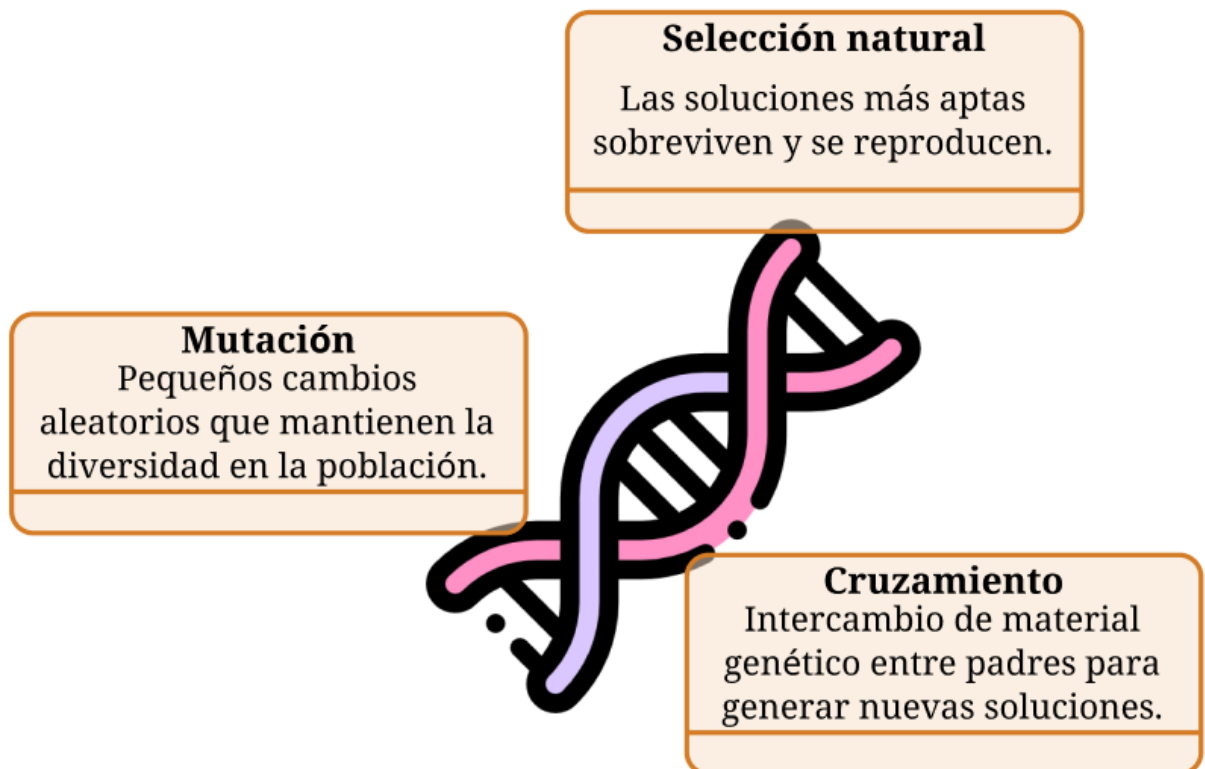
2.2.1. Algoritmos genéticos.

Los Algoritmos Genéticos (AG) se fundamentan en los principios de la evolución biológica darwiniana, emulando procesos naturales para optimizar soluciones. Estos algoritmos operan mediante la selección natural, donde las soluciones con mayor aptitud tienen mayores probabilidades de reproducirse y transmitir sus características. El cruzamiento (crossover) permite la combinación de material genético

entre individuos padres, generando descendencia con rasgos heredados que potencialmente mejoran la calidad de las soluciones. Además, la mutación introduce variaciones aleatorias que preservan la diversidad poblacional y evitan la convergencia prematura. La esencia de los AG radica en evolucionar soluciones de manera análoga a la evolución natural, optimizando iterativamente hacia resultados cada vez más eficientes (Subang et al., 2024) (*The FAIRy Tale of Genetic Algorithms*, 2023).

Figura 2

Principales mecanismos de los algoritmos genéticos: selección natural, mutación y cruzamiento.



Fuente: Elaboración Propia

Estructura Básica del Algoritmo (GA)

Codificación de soluciones (cromosomas)

- Cada solución se representa como un cromosoma por general en forma binaria (01001) pero también números reales o mixtas.

Inicialización de la población

- Se genera aleatoriamente un conjunto de soluciones candidatas.

Evaluación

- Cada cromosoma se evalúa con una función objetivo que mide su calidad.

Selección

- Se eligen los mejores cromosomas para ser padres.
- Métodos comunes: ruleta, torneo, ranking.

Cruzamiento

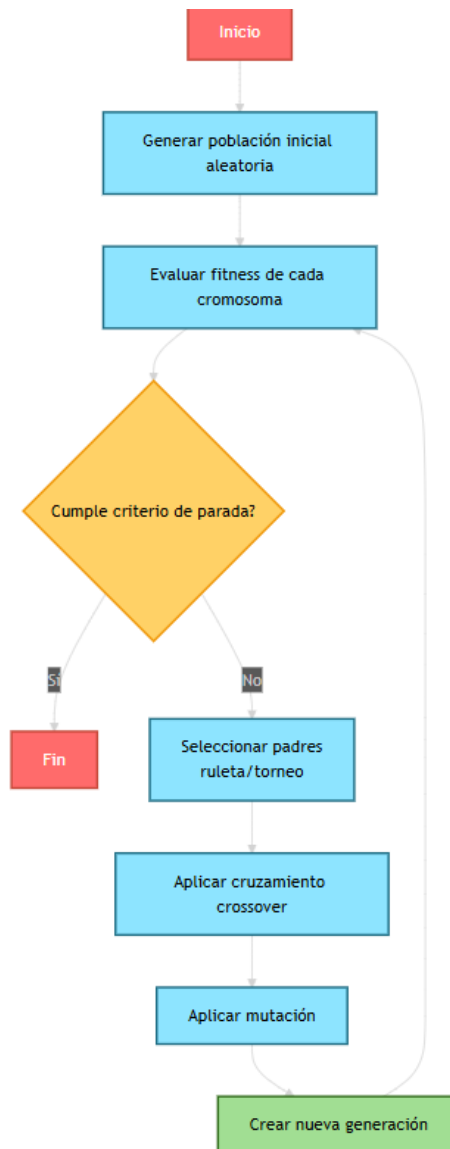
- Combina partes de dos padres para formar hijos.

Mutación

- Se alteran algunos genes aleatoriamente para mantener la diversidad.

Reemplazo y nueva generación

- Los hijos sustituyen parcial o totalmente a los padres.
- El ciclo se repite hasta cumplir un criterio de parada (número de generaciones o calidad deseada).

Figura 3: Diagrama de flujo del ciclo de un Algoritmo Genético.

Fuente: Elaboración Propia.

Ejemplos de implementación

Ahora vamos a explicar cada uno de estos procesos en Python.

Evaluación

En este bloque se evalúa la calidad de cada individuo aplicando la función objetivo (fitness), se identifica el mejor de la generación y luego se aplica la selección por torneo, un mecanismo donde se eligen al azar dos individuos y se conserva el más apto; este proceso se repite hasta formar la nueva lista de padres que participarán en el cruce y la mutación. De esta forma, se garantiza que las soluciones con mejor desempeño tengan más probabilidad de transmitir sus características a la siguiente generación.

```
1 def evaluar(poblacion, funcion_objetivo):  
2  
3     return np.array([funcion_objetivo(ind) for ind in poblacion])
```

Fuente: Elaboración Propia

Cruce

En esta parte se implementa el cruce de un punto (one-point crossover), donde se toman los padres en pares y, con cierta probabilidad (pc), se elige un punto de corte en sus cromosomas; a partir de ese punto se intercambian segmentos para generar dos nuevos hijos que combinan información genética de ambos progenitores. Si no ocurre cruce (cuando el número aleatorio es mayor que pc), los hijos son copias directas de los padres. Este mecanismo asegura diversidad en la población y permite explorar nuevas combinaciones de soluciones.

```
1 def cruce(parents, pc, dim):
2
3     offspring = []
4     pop_size = len(parents)
5     for i in range(0, pop_size, 2):
6         if np.random.rand() < pc:
7             cross_point = np.random.randint(1, dim)
8             child1 = np.concatenate((parents[i][:cross_point], parents[i+1][cross_point:]))
9             child2 = np.concatenate((parents[i+1][:cross_point], parents[i][cross_point:]))
10            offspring.append(child1)
11            offspring.append(child2)
12        else:
13            offspring.append(parents[i])
14            offspring.append(parents[i+1])
15    return np.array(offspring)
```

Fuente: Elaboración Propia

Mutación

En este bloque se aplica la mutación, que introduce pequeñas variaciones aleatorias en los cromosomas para mantener la diversidad genética en la población. Con una probabilidad definida (pm), se selecciona un gen al azar dentro de un individuo (idx) y se modifica sumándole un valor tomado de una distribución normal. Este cambio evita que el algoritmo se estanque en óptimos locales y aumenta la posibilidad de explorar nuevas regiones del espacio de búsqueda.

```
1 def mutacion(offspring, pm, dim):
2
3     for ind in offspring:
4         if np.random.rand() < pm:
5             idx = np.random.randint(0, dim)
6             ind[idx] += np.random.normal()
7     return offspring
```

Fuente: Elaboración Propia

Con todas las funciones creada solo faltaría inicializar variables como la probabilidad de cruce, mutación y número de generaciones:

```
1  n_gen = 50
2  pop_size = 20
3  dim = 2
4  pc, pm = 0.8, 0.1
5
6  # Inicializar población
7  pop = np.random.uniform(-5, 5, (pop_size, dim))
8
9  # ---- Iteraciones ----
10 for gen in range(n_gen):
11     # Evaluación
12     fitness = evaluar(pop, sphere)
13     best_idx = np.argmin(fitness)
14
15     # Selección por torneo
16     parents = []
17     for _ in range(pop_size):
18         i, j = np.random.randint(0, pop_size, 2)
19         parents.append(pop[i] if fitness[i] < fitness[j] else pop[j])
20     parents = np.array(parents)
21
22     # Cruce
23     offspring = cruce(parents, pc, dim)
24
25     # Mutación
26     offspring = mutacion(offspring, pm, dim)
27
28     # Nueva generación
29     pop = offspring
30
```

Fuente: Elaboración Propia

Implementación en R

Utilizando la librería GA , solo debemos definir la función y los parámetros como vemos en el siguiente ejemplo.

```
1 library(GA)
2
3 # Definición de la función
4 sphere <- function(x) {
5   sum(x^2)
6 }
7
8 # Ejecutar GA
9 set.seed(123)
10 result <- ga(type = "real-valued",
11             fitness = function(x) -sphere(x), # GA maximiza por defecto
12             lower = c(-5, -5), upper = c(5, 5),
13             popSize = 20, maxiter = 50,
14             pcrossover = 0.8, pmutation = 0.1)
15
16 summary(result)
17
```

Fuente: Elaboración Propia

2.2.2. Optimización por Colonia de Hormigas (ACO)

La Optimización por Colonia de Hormigas (ACO) está inspirada en el comportamiento de las hormigas reales para encontrar caminos eficientes entre su nido y las fuentes de alimento.

- Las hormigas depositan feromonas en los senderos recorridos.
- Cuanto más exitoso (más corto/eficiente) es un camino, mayor concentración de feromonas recibe.
- Otras hormigas tienden a seguir los caminos con más feromonas, reforzando las mejores rutas.
- Al mismo tiempo, la evaporación de feromonas evita que se sobreexplota un único camino y permite seguir explorando alternativas.

Figura 4

Conducta de las hormigas en la construcción de rutas, inspiración del algoritmo ACO.



Fuente: Elaboración Propia

Estructura Básica del Algoritmo (ACO)

Inicialización

- Se representa el problema como un grafo (nodos y aristas).
- Se asigna una cantidad inicial de feromonas a cada arista.

Construcción de soluciones

Cada hormiga construye un camino paso a paso, eligiendo la siguiente arista con una probabilidad dependiente de dos factores

- Cantidad de feromona (τ) acumulada en la arista.
- Visibilidad heurística (η), normalmente el inverso de la distancia.

Probabilidad de elegir la arista(i, j):

$$P_{ij} = \frac{\tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}}{\sum_{k \in N_i} \tau_{ik}^{\alpha} \cdot \eta_{ik}^{\beta}}$$

α : controla la importancia de la feromona.

β : controla la importancia de la heurística (ejemplo: 1/distancia).

Actualización de feromonas

Después de que todas las hormigas construyen sus caminos, las feromonas se actualizan:

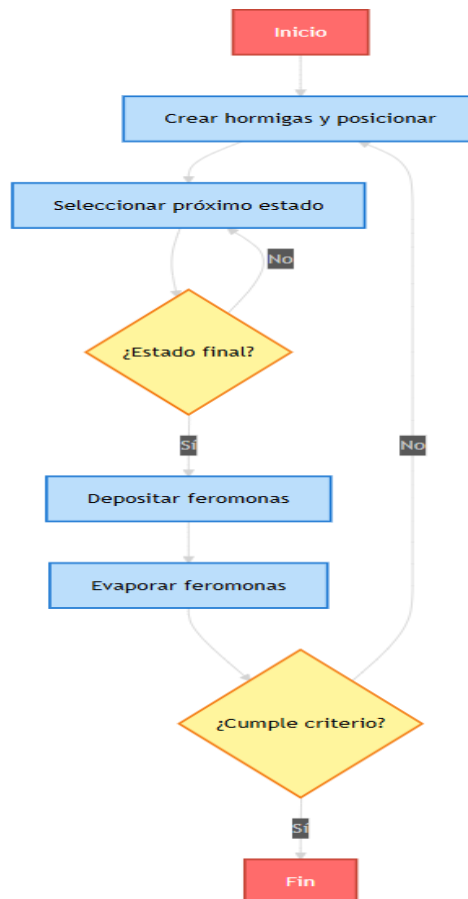
- La evaporación se reduce una fracción de feromona en todas las aristas.
- El refuerzo hace que las mejores soluciones depositen más feromonas en sus aristas.

Iteración

Se repite el proceso de construcción más actualización hasta alcanzar un criterio de parada dado por el número de iteraciones o calidad de solución.

Figura 5

Diagrama de flujo del algoritmo ACO, mostrando sus principales fases iterativas.



Fuente: Elaboración Propia

● Ejemplo de Implementación (ACO)

En este caso se muestra la implementación en R, la librería antcolony permite experimentar de forma práctica con el algoritmo de Optimización por Colonia de Hormigas (ACO) sobre problemas clásicos como el viajante de comercio (TSP), etc.

```
1 library(antcolony)
2
3 # Distancias entre 5 ciudades
4 dist_matrix <- matrix(c(
5   0, 2, 9, 10, 7,
6   2, 0, 6, 4, 3,
7   9, 6, 0, 8, 5,
8   10, 4, 8, 0, 6,
9   7, 3, 5, 6, 0
10  ), nrow=5, byrow=TRUE)
11
12 set.seed(123)
13 result <- antcolony(dist_matrix, iterations = 50, ants = 10)
14
15 print(result$tour) # mejor ruta
16 print(result$length) # distancia total
```

Fuente: Elaboración Propia

2.3. Aplicaciones Comunes de los Algoritmos Bioinspirados.

Los algoritmos Bioinspirados, como la Optimización por Enjambre de Partículas (PSO), los Algoritmos Genéticos (GA) y la Optimización de Colonias de Hormigas (ACO), son herramientas versátiles empleadas en una multitud de campos.

Su principal ventaja radica en abordar problemas de optimización complejos que resultan inalcanzables para las técnicas convencionales. Sus aplicaciones más comunes incluyen:

Ingeniería y Diseño

Se utilizan para optimizar estructuras (reduciendo peso o maximizando rigidez), diseñar circuitos electrónicos y microelectrónicos y crear nuevos materiales como los metamateriales.

Control y Automatización

Son fundamentales para ajustar controladores automáticos (PID) y para planificar la ruta y navegación de robots móviles.

Gestión de Recursos y Operaciones:

Se aplican en la planificación de tareas en sistemas de computación en la nube y en la elaboración de horarios y turnos para empleados, como en el sector sanitario.

Problemas combinatorios

Ofrecen soluciones eficientes a desafíos clásicos como encontrar la ruta más corta, el problema del viajante (TSP), y son vitales en bioinformática para analizar y alinear secuencias genéticas.

Optimización Multiobjetivo

Permiten encontrar equilibrios entre varias metas en conflicto (como coste, rendimiento y riesgo) bajo restricciones específicas, una funcionalidad clave en áreas como la optimización de carteras financieras.

Ciencia de la salud y biología

Se emplean para analizar datos biológicos, ayudar en el diagnóstico médico y procesar la ingente cantidad de información generada por los estudios genómicos.

Redes y Telecomunicaciones

Se usan para optimizar el enrutamiento, distribuir el tráfico de manera eficiente (balanceo de carga) y organizar clusters en redes de sensores inalámbricos.

2.4. Comparación preliminar de desempeño.

Se presenta a continuación un análisis comparativo preliminar sobre el comportamiento de los algoritmos por Enjambre de Partículas (PSO), Algoritmos genéticos (GA) y Optimización por Colonia de Hormigas (ACO) en cuanto a su desempeño, con base en la literatura reciente.

En esta sección examinaremos las ventajas, limitaciones y los contextos específicos en los que un método puede demostrar superioridad sobre los demás.

Tabla 1

Características comparativas de PSO, GA y ACO en distintos aspectos de desempeño.

Aspecto	PSO (Particle Swarm Optimization)	GA (Genetic Algorithms)	ACO (Ant Colony Optimization)
Convergencia / velocidad de obtención de soluciones buenas	Tiende a converger rápido, especialmente en espacios continuos y cuando el ajuste de parámetros es adecuado.	Puede requerir más generaciones/población para converger, sobre todo si la diversidad no se mantiene.	En problemas discretos o combinatorios, ACO puede construir buenas soluciones mediante caminos guiados, pero puede ser más lento y sensible al tamaño del problema.

Algoritmos Bioinspirados PSO

Algoritmos Bioinspirados GA

Aplicaciones

Comparación

Capacidad para evitar óptimos locales

Con mecanismos adecuados (inercia, velocidad, variantes de PSO híbridas), puede evitar óptimos locales, pero suele necesitar cuidados.

Buena capacidad si se usan operadores que introduzcan diversidad (mutación fuerte, cruzamientos variados).

Fuerte en exploración inicial(es) mediante feromonas, pero puede caer en explotación prematura si el refresco de feromonas no está bien diseñado.

Adaptabilidad a diferentes tipos de problemas

Muy eficaz en problemas continuos, de optimización de parámetros; también ha sido adaptado a problemas discretos con variantes.

Altamente versátil: funciona bien para problemas discretos, continuos, multiobjetivo, etc.

Especialmente bueno para problemas combinatorios, rutas, secuenciación, asignaciones (ej. TSP, rutas, alineamientos).

Uso de recursos computacionales / eficiencia

Generalmente menos costoso en cómputo por iteración; fácil de paralelizar.

Dependiente del tamaño de la población y de los operadores; puede requerir mayor esfuerzo si la población es grande.

Puede requerir más cálculos logísticos (actualización de feromonas, búsqueda de caminos, etc.), lo que puede hacer que sea menos eficiente en casos grandes sin optimización. También se han extendido versiones multiobjetivo de ACO; sin embargo, su ajuste de parámetros (feromonas, evaporación, etc.) puede ser más delicado.

Robustez frente a ruido, restricciones, objetivos múltiples

Mantiene buen desempeño si se adaptan restricciones; se han desarrollado versiones multiobjetivo de PSO efectivas.

GA tiene muchas variantes multiobjetivo (por ejemplo NSGA-II, etc.), lo que lo hace útil en contextos con múltiples criterios.

Fuente: Elaboración Propia

Bibliografía

Ahmed, Z., Haron, H., & Al-Tameem, A. (2024). Appropriate Combination of Crossover Operator and Mutation Operator in Genetic Algorithms for the Travelling Salesman Problem. *Computers, Materials & Continua*, 79(2), 2399-2425. <https://doi.org/10.32604/cmc.2024.049704>

Baumann, J., Rutter, I., & Sudholt, D. (2024). Evolutionary Algorithms for One-Sided Bipartite Crossing Minimisation (No. arXiv:2409.15312). arXiv. <https://doi.org/10.48550/arXiv.2409.15312>

Bio-inspired Optimization: Algorithm, Analysis and Scope of Application. (2023). *Artificial Intelligence*. <https://doi.org/10.5772/intechopen.106014>

Fan, S., Wang, R., Song, Y., Crosbee, D., & Su, K. (2025). Nature-inspired adaptive differential evolution: A unified meta-heuristic framework for complex engineering optimisation and UAV path planning. *Results in Engineering*, 27, 106530. <https://doi.org/10.1016/j.rineng.2025.106530>

Frank, S. A. (2023). Robustness and complexity. *Cell Systems*, 14(12), 1015-1020. <https://doi.org/10.1016/j.cels.2023.11.003>

Game, P., Vaze, V., & Emmanuel, M. (2020). Bio-inspired Optimization: metaheuristic algorithms for optimization. arXiv: Neural and Evolutionary Computing. <http://arxiv.org/pdf/2003.11637.pdf>

Khalid, A. M., Hosny, K. M., & Mirjalili, S. (2022). COVIDOA: A novel evolutionary optimization algorithm based on coronavirus disease replication lifecycle. *Neural Computing and Applications*, 34(24), 22465-22492. <https://doi.org/10.1007/s00521-022-07639-x>.

Subang, K. N., Balaba, E. I., & Agoylo, J. C. (2024). Optimizing Course Scheduling with Genetic Algorithms: A Dynamic Approach. *SAR Journal*, 296–302. <https://doi.org/10.18421/sar74-02>

The FAIRy Tale of Genetic Algorithms. (2023). <https://doi.org/10.48550/arxiv.2305.00238>

Zeb, A., Din, F., Fayaz, M., Mehmood, G., & Zamli, K. Z. (2023). A Systematic Literature Review on Robust Swarm Intelligence Algorithms in Search-Based Software Engineering. *Complexity*, 2023, 1-22. <https://doi.org/10.1155/2023/4577581>